

SOLID + Patrones de Diseño — versión mínima con ejemplos en Java

SOLID		
	Principio	En una línea
S	Single Responsibility	Una clase, una razón para cambiar.
O	Open/Closed	Abierta a extensión, cerrada a modificación.
L	Liskov Substitution	Toda subclase debe poder reemplazar a su base sin romper nada.
I	Interface Segregation	Muchas interfaces chicas y específicas mejor que una gorda.
D	Dependency Inversion	Dependé de abstracciones, no de implementaciones concretas.

S — Single Responsibility

```
// MAL: La clase calcula, persiste y notifica: tres motivos de cambio
class Prestamo { BigDecimal calcularCuota(){...} void guardar(){...} void enviarMail(){...} }

// BIEN: Cada responsabilidad en su lugar
class CalculadorCuota { BigDecimal calcular(Prestamo p) {...} }
interface PrestamoRepository { void guardar(Prestamo p); }
class NotificadorPrestamo { void notificar(Prestamo p) {...} }
```

O — Open/Closed

```
// MAL: Cada medio de pago nuevo obliga a tocar este switch
BigDecimal comision(String tipo) {
    if ("TARJETA".equals(tipo)) return ...; else if ("DEBITO".equals(tipo)) return ...;
}

// BIEN: Agregar un medio de pago = agregar una clase, no modificar las existentes
interface Comision { BigDecimal calcular(BigDecimal monto); }
class ComisionTarjeta implements Comision { public BigDecimal calcular(BigDecimal m){ return m.multiply(new BigDecimal("0.03")); } }
class ComisionDebito implements Comision { public BigDecimal calcular(BigDecimal m){ return BigDecimal.ZERO; } }
```

L — Liskov Substitution

```
// MAL: La subclase rompe el contrato de la base
class CuentaAhorro extends Cuenta { void extraer(BigDecimal m){ throw new UnsupportedOperationException(); } }

// BIEN: Modelar la capacidad real en vez de heredar de más
interface Cuenta { BigDecimal saldo(); }
interface CuentaExtraible extends Cuenta { void extraer(BigDecimal m); }
```

Señales de violación: `throw new UnsupportedOperationException()`, precondiciones más estrictas o postcondiciones más débiles que la base, y `if (x instanceof SubTipo)` en el código cliente.

I — Interface Segregation

```
// MAL: El cliente implementa métodos que no usa
interface Repositorio { void guardar(X x); void borrar(Long id); void exportarCsv(); void reindexar(); }

// BIEN
interface Lector<T> { Optional<T> buscar(Long id); }
interface Escritor<T> { void guardar(T t); }
```

D — Dependency Inversion

```
// MAL: El servicio (alto nivel) depende de una implementación concreta (bajo nivel)
class ClienteService { private final OracleClienteDao dao = new OracleClienteDao(); }

// BIEN: Ambos dependen de la abstracción; la implementación se inyecta
@Service
class ClienteService {
    private final ClienteRepository repo; // interfaz
    ClienteService(ClienteRepository repo) { this.repo = repo; } // inyección por constructor
}
```

Bonus: **KISS**, **DRY**, **YAGNI** y *composición sobre herencia*. SOLID es una guía, no un dogma: aplícalo donde hay cambio real, no para inflar la cantidad de clases.

Singleton

```
public enum Config {                                // la forma más segura en Java
    INSTANCIA;
    private final Properties props = cargar();
    public String get(String k) { return props.getProperty(k); }
}
```

Cuándo: una única instancia compartida. Ojo: en Spring, un `@Bean` ya es singleton; evitá el patrón manual (dificulta los tests).

Factory Method

```
interface Notificador { void enviar(String msg); }

class NotificadorFactory {
    static Notificador crear(Canal canal) {
        return switch (canal) {
            case MAIL -> new NotificadorMail();
            case SMS -> new NotificadorSms();
            case PUSH -> new NotificadorPush();
        };
    }
}
```

Cuándo: decidir en runtime qué implementación instanciar, sin que el cliente conozca las clases concretas.

Abstract Factory

```
interface FabricaUI { Boton boton(); Checkbox checkbox(); }
class FabricaOscura implements FabricaUI { public Boton boton(){return new BotonOscuro();} public Checkbox checkbox(){return new
CheckboxOscuro();} }
```

Cuándo: crear **familias** de objetos relacionados que deben ser coherentes entre sí.

Builder

```
public class Transferencia {
    private final String origen, destino; private final BigDecimal monto; private final String concepto;
    private Transferencia(Builder b) { this.origen=b.origen; this.destino=b.destino; this.monto=b.monto; this.concepto=b.concepto; }
    public static Builder builder() { return new Builder(); }

    public static class Builder {
        private String origen, destino, concepto; private BigDecimal monto;
        public Builder origen(String o){ this.origen=o; return this; }
        public Builder destino(String d){ this.destino=d; return this; }
        public Builder monto(BigDecimal m){ this.monto=m; return this; }
        public Transferencia build(){ Objects.requireNonNull(origen); return new Transferencia(this); }
    }
}

// Transferencia.builder().origen("123").destino("456").monto(TEN).build();
```

Cuándo: muchos parámetros, varios opcionales, y querés objetos inmutables. Lombok: `@Builder`.

Prototype

```
record Plantilla(String nombre, List<String> campos) {
    Plantilla copiaCon(String nuevoNombre) { return new Plantilla(nuevoNombre, List.copyOf(campos)); }
}
```

Cuándo: clonar un objeto configurado sale más barato que construirlo de cero.

Patrones estructurales

Adapter

```
// Adaptar una API externa/legacy a la interfaz que espera mi dominio
class ServicioCotizacionAdapter implements Cotizador {
    private final LegacyFxClient legacy;
    public BigDecimal cotizar(String par) { return BigDecimal.valueOf(legacy.getRate(par.replace("/", ""))); }
}
```

Decorator

```
interface Repositorio { Cliente buscar(Long id); }

class RepositorioConCache implements Repositorio { // agrega comportamiento sin tocar el original
    private final Repositorio delegado;
    private final Map<Long, Cliente> cache = new ConcurrentHashMap<>();
    public Cliente buscar(Long id) { return cache.computeIfAbsent(id, delegado::buscar); }
}
```

En la JDK: `BufferedReader(new FileReader(f)), Collections.unmodifiableList(...)`.

Facade

```
// Una entrada simple a un subsistema complejo
class AltaClienteFacade {
    void alta(ClienteDTO dto) { validador.validar(dto); var c = repo.guardar(mapper.aEntidad(dto));
                                scoring.evaluar(c); notificador.bienvenida(c); }
}
```

Proxy

```
// Controla el acceso al objeto real: lazy loading, seguridad, logging, remoto
class RepositorioAuditado implements Repositorio {
    private final Repositorio real;
    public Cliente buscar(Long id) { auditoria.registrar("buscar", id); return real.buscar(id); }
}
```

Nota: Spring usa proxies dinámicos para `@Transactional`, `@Cacheable` y AOP.

Composite

```
interface Nodo { BigDecimal total(); }
record Item(BigDecimal precio) implements Nodo { public BigDecimal total(){ return precio; } }
record Paquete(List<Nodo> hijos) implements Nodo {
    public BigDecimal total(){ return hijos.stream().map(Nodo::total).reduce(BigDecimal.ZERO, BigDecimal::add); }
}
```

Cuándo: estructuras de árbol donde hoja y compuesto se tratan igual.

Patrones de comportamiento

Strategy — el más usado

```
interface EstrategiaInteres { BigDecimal calcular(BigDecimal capital, int dias); }

@Service
class LiquidadorService {
    private final Map<TipoCuenta, EstrategiaInteres> estrategias; // Spring inyecta todas las impl.
    BigDecimal liquidar(Cuenta c) { return estrategias.get(c.getTipo()).calcular(c.getSaldo(), 30); }
}
```

Cuándo: varios algoritmos intercambiables. Mata los `if/else` largos.

Observer

```
// Spring lo trae de fábrica
class ClienteCreado extends ApplicationEvent { ... }
publisher.publishEvent(new ClienteCreado(cliente));

@EventListener
void onClienteCreado(ClienteCreado e) { mailService.bienvenida(e.getCliente()); }
```

Template Method

```
abstract class ProcesoBatch {
    public final void ejecutar() { abrir(); procesar(); cerrar(); } // el esqueleto es final
    protected abstract void procesar(); // los hijos completan el hueco
    private void abrir() {...} private void cerrar() {...}
}
```

Command

```
interface Comando { void ejecutar(); void deshacer(); }
record Acreditar(Cuenta c, BigDecimal monto) implements Comando {
    public void ejecutar(){ c.acreditar(monto); }
    public void deshacer(){ c.debitar(monto); }
}
// Permite encolar, loguear, reintentar y deshacer operaciones
```

State

```
interface EstadoPedido { EstadoPedido siguiente(Pedido p); }
// PENDIENTE -> CONFIRMADO -> ENVIADO -> ENTREGADO. Cada estado sabe a cuál puede pasar,
// en vez de un switch gigante sobre un enum.
```

Chain of Responsibility

```
interface Filtro { boolean aplica(Solicitud s); void manejar(Solicitud s, Cadena c); }
// Cada eslabón decide si procesa o delega. Ej.: filtros de Servlet, interceptors, middlewares de validación.
```

Iterator

```
for (Cliente c : clientes) { ... } // Iterable/Iterator ya implementado por la JDK
```

Cuál elegir — guía rápida

Síntoma en el código	Patrón
<code>if/else</code> o <code>switch</code> largo por tipo	Strategy / Factory / polimorfismo
Constructor con 8 parámetros	Builder
Necesito agregar logging, cache o seguridad sin tocar la clase	Decorator / Proxy
Una API externa no encaja con mi dominio	Adapter
Varios pasos fijos con detalles variables	Template Method
Algo debe reaccionar cuando pasa X	Observer / eventos
El objeto cambia de comportamiento según su situación	State
Necesito deshacer / reintentar operaciones	Command
El cliente debe coordinar 5 servicios para una tarea	Facade
Estructura de árbol tratada uniformemente	Composite